

ArrayList Interview Handbook

Written in the same learning style used during our discussions: WHY → HOW → WHEN → INTERVIEW.

1. Why was ArrayList created?

Arrays are very fast because you can directly access an element using its index. The problem is that arrays have a fixed size. Imagine creating an array of size 10. When the 11th element arrives, the array cannot grow. Java introduced **ArrayList** to solve this problem. Think of it as a normal array that automatically grows when it becomes full.

Real Life Analogy

Imagine a bookshelf with 10 slots. When all slots are occupied and you buy one more book, you don't throw the shelf away. You buy a bigger shelf, move all books to it, and continue using it. ArrayList behaves the same way.

2. Internal Working

Internally ArrayList stores data inside an **Object[]** array. When there is free space, the new element is placed at the next index. When the array is full: 1. Java creates a larger array. 2. Copies all existing elements. 3. Inserts the new element. The old array becomes eligible for garbage collection.

Internal Picture

Before resize:
Index: 0 1 2 3
Data : A B C D

After adding one more element:
Create bigger array → Copy A,B,C,D → Insert E.

3. Why is get(index) O(1)?

Because the backing structure is an array. Java can directly jump to index 5000 without checking indexes 0..4999. That is called random access.

4. Why is insertion in the middle slow?

Suppose the list is [10,20,30,40]. If you insert 15 at index 1, Java must shift 20,30 and 40 one place to the right before inserting 15. More shifting means more work, therefore O(n).

5. Time Complexity

Operation	Complexity	Reason
get(index)	O(1)	Direct array access
add(end)	Amortized O(1)	Usually append; resize only occasionally
add(index)	O(n)	Elements shift
remove(index)	O(n)	Elements shift left
contains()	O(n)	Checks one by one

6. When should you use ArrayList?

Use ArrayList when: • You frequently read data by index. • Most insertions happen at the end. • You don't often insert/remove from the middle.

7. Interview Questions

Q: Why is `add()` called amortized $O(1)$?

A: Most additions simply place the element at the next free index. Only occasionally does resizing occur, so the average cost remains $O(1)$.

Q: Why isn't middle insertion $O(1)$?

A: Because all later elements must shift.

Q: ArrayList or LinkedList for random access?

A: ArrayList because it is backed by an array.

8. Common Mistakes

- Thinking ArrayList uses LinkedList internally. It does not.
- Thinking `resize` happens on every `add()`. It happens only when full.
- Assuming `add(index)` is fast. It requires shifting.

9. Remember Forever

Remember these four lines:

- ArrayList = Dynamic Array.
- Fast random access.
- Automatic `resize` when full.
- Middle insertion/removal is slow because of shifting.